gaugefit — Bias Characteristics

What gaugefit gets wrong, measured, with the conditions attached. Current as of version 0.1.0.

A Korean edition of this document is at `docs/bias.ko.md`; the content is the same.

Commercial caliper tools quote a subpixel accuracy and stop there. That number is only meaningful with the conditions beside it, and the conditions are where the interesting failures live. This document is the conditions. It exists so that a disputed measurement can be settled by pointing at a row instead of arguing about a claim.

Everything here comes from **analytically anti-aliased synthetic images**: the coverage of each pixel by the shape is integrated in closed form, so the true edge position is known to machine precision. The renderer itself was checked against three independent references — polygon clipping, 2-D Gauss–Legendre quadrature, and a fine-grid sum — agreeing to about 1e-12. The raw sweep ships with the package as `gaugefit/data/bias_table.csv`, and the error budget reads from it, so these numbers are not decoration: they are what `caliper_budget()` returns.

Baseline conditions unless stated: contrast 0.70 (fg 0.85 / bg 0.15), lens PSF σ 0.8 px, ROI length 40 px, projection width 20 px, caliper σ 1.5 px, noise-free.

Three metrics, kept separate:

metric	meaning
offset	a constant shift — calibration absorbs this, so it is the least interesting
pp	peak-to-peak of the bias as the phase varies — *cannot* be calibrated away, and is the practical limit
harm1	the one-cycle-per-pixel component of pp, the fingerprint of resampling bias

---

1. There are two different biases, on two different axes

The first thing the harness established, and it changes how everything else is measured.

When the ROI grid lands on pixel centres, interpolation becomes the identity, and every resampling method returns exactly the same numbers. (`bspline3_box` is the one exception — it is not an interpolator but a reconstruction that deconvolves the pixel box, so it changes values even at sample points.) Therefore:

- move the **edge** within a pixel, with the ROI aligned → you are measuring the **subpixel estimator**
- move the **ROI** against the pixel grid → you are measuring the **resampler**

Mixing the two hides which one is responsible. Every table below is one or the other, never both.

---

2. Subpixel estimator: `gauss3` beats `parabola3` by two orders of magnitude

Edge-phase sweep, ROI aligned (interpolation is the identity), noise-free.

subpixel	σ	offset	pp	harm1
parabola3	1.0	–0.00000	0.05446	0.02726

subpixel	σ	offset	pp	harm1
parabola3	1.5	0.00000	0.03222	0.01616
parabola3	2.0	0.00000	0.02034	0.01021
parabola3	3.0	0.00000	0.00989	0.00497
gauss3	1.0	0.00000	**0.00106**	0.00053
gauss3	**1.5**	0.00000	**0.00009**	0.00004
gauss3	2.0	0.00000	0.00002	0.00001
gauss3	3.0	0.00000	0.00000	0.00000
centroid	1.0	–0.00000	0.09035	0.04618
centroid	1.5	0.00000	0.12154	0.06206
centroid	2.0	0.00000	0.07234	0.03717
centroid	3.0	0.00000	0.05987	0.03081

At σ = 1.5 the ratio is **350×**. The reason is not subtle: a Gaussian-blurred edge has a Gaussian first derivative, and a log-parabola through three samples is exact for a Gaussian while a plain parabola is not. `parabola3` is fitting the wrong function.

`centroid` gets *worse* as σ rises, because a wider kernel drags more of the neighbouring structure into the moment.

This is why `subpixel="gauss3"` is the default. There is no condition in the sweep where `parabola3` wins.

---

3. Resampling: `bicubic` is not better than `bilinear`. B-spline is

ROI-phase sweep, edge fixed, noise-free. σ here is the lens PSF.

interp	psf σ	offset	pp	harm1
bilinear	0.5	–0.00131	0.03428	0.01734
bilinear	0.8	0.00026	0.03035	0.01535
bilinear	1.5	0.00012	0.02017	0.01019
bicubic	0.5	–0.00135	0.03562	0.01803
bicubic	0.8	0.00030	0.03146	0.01593
bicubic	1.5	0.00013	0.02073	0.01047
bspline3	0.5	–0.00173	**0.00367**	0.00184
bspline3	**0.8**	0.00005	**0.00302**	0.00151
bspline3	1.5	0.00002	**0.00125**	0.00062

`bicubic` is very slightly *worse* than `bilinear`, consistently. It costs four times the taps and buys nothing. This is worth saying plainly because "use bicubic for accuracy" is common advice and it is wrong here.

`bspline3` is **10× better**, because cubic B-spline interpolation with the prefilter is interpolating rather than approximating: it reproduces the samples exactly and its transfer function is much flatter across the

passband.

This is why `interp="bspline3"` is the default, and why there is no "fast bicubic mode" — it would be slower than bilinear and no more accurate. If you need speed, use `bilinear` and accept about 0.03 px.

---

4. An axis-aligned edge is the worst case

The one that surprises people. A perfectly horizontal or vertical edge is the *hardest* case for resampling bias, not the easiest.

ROI-phase sweep, psf σ 0.8, by edge angle:

interp	angle	offset	pp	harm1
bilinear	0°	0.00026	0.03035	0.01535
bilinear	5°	0.00002	0.00400	0.00198
bilinear	15°	0.00057	0.00133	0.00042
bilinear	30°	0.00036	0.01034	0.00020
bilinear	45°	–0.00006	0.00217	0.00106
bicubic	0°	0.00030	0.03146	0.01593
bicubic	15°	0.00007	0.00090	0.00044
bspline3	**0°**	0.00005	**0.00302**	0.00151
bspline3	5°	–0.00001	**0.00043**	0.00021
bspline3	15°	0.00000	**0.00012**	0.00006
bspline3	30°	0.00002	0.00007	0.00001
bspline3	45°	–0.00003	0.00026	0.00013

At 0° every row of the ROI samples the *same* subpixel phase, so the resampling error is identical everywhere and the projection averages nothing away. Tilt by 5° and the rows sample different phases, which averages the error down by an order of magnitude.

Practical consequence: if you are free to choose, mount so the critical edges are a few degrees off axis. If you are not, budget the 0° row, not the 15° one.

---

5. Projection width × misalignment: the product is what matters

If the ROI axis is misaligned from the edge normal by Δ, each row crosses the edge at a different `t`, and the projection smears the profile. The dominant quantity is not the width and not the angle but `W·tanΔ`, the total spread across the ROI.

Neither alone predicts the damage; the product does. ROI-phase sweep, `bspline3`:

Δ	W	W·tanΔ	offset	pp
0°	4	0.00	0.00005	0.00302
0°	20	0.00	0.00005	0.00302
0°	50	0.00	0.00005	0.00302

Δ	W	W·tanΔ	offset	pp
5°	4	0.35	0.00005	0.00267
5°	20	1.75	–0.00004	0.00093
5°	50	4.37	0.00000	0.00329
10°	20	3.53	0.00002	0.00130
10°	50	8.82	0.00002	**0.03391**
20°	20	7.28	–0.00001	0.01787
20°	50	18.20	0.00301	**0.50364**

Notice the 5°/W=20 row: a *small* tilt is mildly helpful, because it decorrelates the row phases the way §4 describes. The damage starts when the smear approaches the kernel.

As the smear grows the profile stops looking Gaussian, and that is exactly when `gauss3` loses its advantage — its whole basis is that a blurred edge has a Gaussian derivative. Holding W = 50 and varying Δ:

subpixel	W·tanΔ	pp
parabola3	0.00	0.03329
parabola3	4.37	0.01536
parabola3	8.82	0.03092
parabola3	18.20	0.49930
gauss3	0.00	**0.00302**
gauss3	4.37	**0.00329**
gauss3	8.82	0.03388
gauss3	18.20	0.49930

`gauss3` keeps its 10× lead up to about `W·tanΔ ≈ 4 px`, matches `parabola3` at 9 px, and at 18 px both collapse to half a pixel. Hence the rule:

```
W · tanΔ  <  3σ
```

Inside that, the smear folds into the effective σ and costs precision but not accuracy. gaugefit measures it for you: with `tilt_bands > 0` the ROI is split along the projection direction, the edge is located in each band, and `diag` reports `tilt_deg`, `smear` and `effective_sigma`. The `TILTED` flag fires when `smear > 3σ`.

The trade is real in both directions. Widening `W` reduces noise as `1/√W` — the cheapest precision available — as long as the edge stays straight and aligned inside the ROI.

---

6. Curvature bias comes from the blur and the width, not from σ

Measuring a curved edge with a **straight** ROI reads short. The measured radius is biased by

```
Δr  ≈  – (σ_psf² + W²/12) / (2R)
```

Two things are worth noticing.

It does not depend on the caliper σ. Raising the smoothing does not make it worse. The `W²/12` term is the chord effect — the ROI averages across an arc and reads the chord — and `σ_psf²` is the 2-D lens blur pulling the apparent edge inward on a convex boundary. The caliper's own 1-D smoothing is along the measurement axis and does not enter.

It is not small, and the dominant term is W, not the optics. Measured against the formula, residuals at the 1e-4 px level:

R	σ_psf	W	measured	predicted	resid
20	0.0	5	–0.05196	–0.05208	0.00012
20	0.0	10	–0.20926	–0.20833	–0.00092
20	1.2	5	–0.08816	–0.08808	–0.00008
40	0.8	10	–0.11211	–0.11217	0.00006
60	1.5	20	–0.29525	–0.29653	0.00128
80	0.8	1	–0.00450	–0.00452	0.00002

The law is settled well enough to correct analytically — no lookup table is needed. At R = 20 with W = 20 a linear ROI reads **0.81 px** short on the radius. On a hole calibrated at 40 px/mm that is 20 µm of systematic error that no amount of averaging removes.

Unwrapping really does remove it. Same projection width, linear ROI against annular:

R	W	linear	annular	chord term `–W²/24R`
20	5	–0.06811	–0.01806	–0.05208
20	20	–0.81135	**–0.01820**	–0.83333
40	20	–0.42144	**–0.00903**	–0.41667
60	30	–0.61519	**–0.00602**	–0.62500

The linear ROI is displaced by exactly the chord term; in the annular ROI that term is simply gone — 45× better at R = 20, W = 20. What remains, –0.018 px, is the optical term `–σ_psf²/2R = –0.016`. Set σ_psf to zero and the annular residual falls to –0.002 px.

What gaugefit does about it. `CircleFinder` uses **annular** calipers, which unwrap the ROI along the arc, so the `W²/12` term does not exist. The remaining `σ_psf²/(2R)` term is corrected automatically: σ_psf is backed out of the measured total edge width using

```
σ_psf² = σ_total² − σ_caliper² − du²/12
```

where `σ_total` comes from the curvature of the log-parabola fitted at the derivative peak. The correction applied is reported in `diag["curvature_correction"]`, and the σ_psf used in `diag["psf_sigma"]`. Pass `psf_sigma=` to override it, or `curvature_correction=False` to turn it off.

If you must use linear ROIs on a curved edge, apply the formula yourself. Do not assume it is negligible.

---

7. Adjacent edges interfere out to 5σ, not 3σ

The usual figure is that a Gaussian derivative kernel is done at 3σ. For *detection* that is true. For *unbiased position* it is not: the tails of the neighbouring peak still pull, and the pull is systematic.

A slit sweep, measuring the width error against the gap in units of σ:

gap / σ	width error (px)	missed
2	1.14 – 1.84	some
3	0.50 – 1.09	none
4	0.04 – 0.13	none
5	**0.002 – 0.019**	none
6	< 0.001	none
8	0.000	none

The error falls below 0.01 px only at about **gap = 5**σ. At 4σ it is 0.04 px, at 3σ half a pixel, at 2σ more than a pixel and detection starts failing outright. The peaks attract each other, so a narrow slit reads *narrower* than it is — a systematic error in a direction that flatters nobody.

gaugefit raises `EDGES_TOO_CLOSE` on the 5σ criterion, not the 3σ one. When it fires:

- lower `sigma` if the noise allows — the interference scales with the kernel, not the edge
- or measure the two edges with separate calipers, each seeing only one
- or accept it and correct, if the geometry is fixed and you can characterise it

---

8. Noise: the estimators do not differ

Repeatability (1σ over repeats) against phase-dependent bias (pp), by estimator:

subpixel	SNR	pp (bias)	repeat (noise)
parabola3	40 dB	0.03697	0.00475
parabola3	30 dB	0.04716	0.01502
parabola3	20 dB	0.08471	0.04714
gauss3	40 dB	**0.00657**	0.00474
gauss3	30 dB	**0.01782**	0.01499
gauss3	20 dB	0.05625	0.04705
zero_cross	40 dB	0.03563	0.00474
zero_cross	30 dB	0.04415	0.01501
zero_cross	20 dB	0.07178	0.04716

The `repeat` column is the same to three digits across all three estimators. Taking a logarithm does not make `gauss3` fragile under noise. Noise sets a floor of roughly

```
σ_position  ∝  10^(−SNR/20)
```

and the estimator does not move that floor — it only decides how much *systematic* error sits on top of it. Below about 20 dB the noise swamps the systematic term and the choice stops mattering at all.

Which means: **do not choose the estimator to reduce noise, and do not increase averaging to reduce bias.** They are separate problems with separate remedies. Noise → more width, more σ, better lighting. Bias → better interpolation, a tilted edge, the right estimator.

This separation is the reason `caliper_budget()` reports `kind="random"` and `kind="systematic"` terms apart instead of a single number.

---

9. Cell integration and profile oversampling

`super_u` and `super_v` subdivide each ROI cell and integrate (Gauss–Legendre) instead of point-sampling. `du` steps the profile at sub-pixel intervals.

interp	super_u	du	offset	pp	harm1
bilinear	1	1.0	0.00026	0.03032	0.01535
bilinear	1	0.5	–0.00470	0.00775	**0.00000**
bilinear	4	1.0	0.00017	0.01691	0.00868
bilinear	4	0.5	–0.00412	0.00411	**0.00000**
bspline3	1	1.0	0.00005	0.00302	0.00151
bspline3	1	0.5	–0.00334	0.00114	**0.00000**
bspline3	4	1.0	0.00001	0.00112	0.00056
bspline3	4	0.5	–0.00306	0.00101	**0.00000**

- `super_u=4` cuts pp by about 2.7× (bspline3 0.0030 → 0.0011), at proportional cost in time.
- `du=0.5` all but eliminates the periodic component (harm1 → 0.000) in exchange for a small *global* offset. That is a good trade, because a global offset is exactly what calibration absorbs and a periodic one is not.

A related hypothesis the sweep answered both ways: **area integration as a reconstruction method (`bspline3_box`) buys nothing, while cell integration along the profile (`super_u`) does.** They sound like the same idea and are not.

---

10. What you may claim

Putting it together, for the default configuration (`bspline3` + `gauss3`, σ 1.5, W 20 px, lens PSF σ 0.8):

condition	phase-dependent bias (pp)
straight edge, 15° off axis, noise-free	**< 0.001 px**
straight edge, axis-aligned, noise-free	**< 0.005 px**
straight edge, axis-aligned, SNR 35 dB	≈ **0.01 px** (noise dominates)
curved edge, annular ROI, corrected	**< 0.01 px** on radius
curved edge, linear ROI, uncorrected	`(σ_psf² + W²/12)/2R` — *state it, do not hide it*

The honest headline is **±0.02 px for θ < 15°, σ = 1.5, SNR > 30 dB, straight edges, no neighbouring edge within 5σ** — and every one of those conditions is doing work.

Reproduce any of it with `python -m synth.report` in the source tree; the full report with curves is `01_python_prototype/BIAS_REPORT.md`.